

**FACULTY OF
ENGINEERING AND
TECHNOLOGY
(FET)
PO BOX 63**



**PEACE-WORK-
FATHERLAND**

BUEA, SOUTH WEST REGION

Task 5: UI Design and Implementation.

COURSE TITLE: Internet Programming & Mobile Programming

COURSE INSTRUCTOR: Dr. Nkemeni Valery

**SUBMITTED BY
GROUP 3 MEMBERS**

NAME	MAT
KONEH CATHERINE MBEH	FE23A076
MOFA DIVINE ESIMO	FE23A095
TUBE NDIANG MARY SHARON	FE23A051
AGBORNJA BESONG AKPAK DIVINE	FE23A006
OBIOMA OBI FODJO CAREL	FE23A133

DEPARTMENT OF COMPUTER ENGINEERING

Table of Contents	
Table of Contents	2
1. Executive Summary	3
2. Technical Achievement Highlights	4
2.1 Technology Stack Overview	4
2.2 MonitoringService — Cellular Telemetry Acquisition.....	4
2.3 Local SQLite Data Persistence	6
2.4 Cloud Synchronisation Readiness — is_synced Flag	7
2.5 Screen Implementation Summary	7
3. Visual Appendix — Proof of Work	8
Figure 1: Dashboard Interface	8
Figure 2: Map View Integration	9
Figure 3: Data Persistence Log	10
Figure 4: System Settings & Configuration	11
Figure 5: Notification & Search UX	12
Figure 6: Premium & Trial Modules	13
Figure 7: Protocol Documentation View	14
Figure 8: Profile Management UI	14
Figure 9: Application Help & Support	15
Figure 10: Dark Mode Aesthetic Verification	15
4. Roadmap for Task 6 — Backend Development	16
4.1 Backend Architecture Overview	16
4.2 Primary API Endpoint — POST /v1/telemetry	16
4.3 Analytics Engine — Coverage Hole Detection.....	17
4.4 Task 6 Milestone Timeline	17
5. Conclusion	18

1. Executive Summary

This report formally documents the successful completion of Task 5 UI Design and Frontend Implementation for the NetSense Network Crowdsensing Platform, developed as part of the CEF440 Internet and Mobile Programming course at the University of Buea.

The NetSense mobile application has been fully implemented as a cross-platform solution targeting Android and iOS, leveraging React Native and the Expo managed workflow as the foundational development framework. The user interface layer employs NativeWind a utility-first styling library that brings Tailwind CSS semantics to React Native enabling the delivery of a consistent, responsive, and visually cohesive design across both target platforms without requiring platform-specific styling overrides.

The implementation phase represents the critical transition from the static artefacts produced in Task 4 (system modelling and design) to a fully interactive and functional mobile application. Wireframes and screen flows originally authored in Figma were systematically translated into production-quality React Native component trees, faithfully preserving the visual identity, interaction model, and information architecture defined during the design phase. All ten primary application screens spanning the real-time monitoring dashboard, GIS-based map view, data persistence logs, settings and configuration panels, notification and search interfaces, premium service modules, protocol documentation viewer, user profile management, help and support screens, and dark-mode aesthetic verification have been implemented, tested on-device, and are confirmed functional.

Beyond the user interface, the frontend implementation encompasses the complete client-side data pipeline: real-time cellular telemetry acquisition via the device's native telephony APIs, GPS coordinate capture with appropriate permission handling, local data persistence using an embedded SQLite database, and cloud synchronisation readiness through an `is_synced` flag mechanism. These capabilities collectively position the NetSense frontend as a fully self-contained, offline-resilient drive-test data collection client, ready for integration with the backend analytics platform to be developed in Task 6.

2. Technical Achievement Highlights

2.1 Technology Stack Overview

Component	Technology	Purpose
Mobile Framework	React Native + Expo SDK 51	Cross-platform iOS/Android app runtime
Styling Engine	NativeWind (Tailwind CSS)	Utility-first responsive UI styling
Navigation	Expo Router (file-based)	Stack and tab-based screen navigation
Local Storage	expo-sqlite / SQLite	Offline measurement persistence
Telemetry APIs	expo-cellular / react-native-cellular-info	RSSI, RSRP, RSRQ, SINR capture
Location Services	expo-location	GPS coordinate and accuracy capture
Design Source	Figma (handoff)	Screen mockups and design tokens
State Management	React Context + useReducer	Application-wide session state

2.2 MonitoringService — Cellular Telemetry Acquisition

The MonitoringService constitutes the core data collection engine of the NetSense application. It is implemented as a singleton service class that interfaces directly with the device's native telephony APIs via the expo-cellular Expo module, augmented by react-native-cellular-info for access to granular LTE signal metrics not exposed through the standard Expo API surface.

The service captures the following signal parameters at a configurable polling interval (default: 5 seconds):

Parameter	Unit	API Source	Description
RSSI	dBm	CellularInfo.getSignalStrength()	Received Signal Strength Indicator

			overall received power
RSRP	dBm	CellularInfo.getLteInfo()	Reference Signal Received Power LTE pilot signal strength
RSRQ	dB	CellularInfo.getLteInfo()	Reference Signal Received Quality signal quality relative to interference
SINR	dB	CellularInfo.getLteInfo()	Signal-to-Interference-plus-Noise Ratio link quality indicator
Cell ID	Integer	Cellular.getCellularGenerationAsync()	Serving cell global identifier
RAT Type	Enum	Cellular.getCellularGenerationAsync()	Radio Access Technology (2G/3G/4G/5G)

The service implementation follows a reactive architecture pattern. A setInterval-based polling loop dispatches measurement records to a shared React Context store, triggering UI re-renders in the live monitoring screen without blocking the JavaScript thread. Error boundaries are implemented to handle cases where the device denies telephony permissions or returns null values for parameters unavailable on the current network generation.

The representative MonitoringService initialisation flow is structured as follows:

```
class MonitoringService {
  constructor(db, locationService) {
    this.db = db;
    this.locationService = locationService;
    this.intervalId = null;
    this.pollingIntervalMs = 5000; // configurable via Settings
  }

  async startMonitoring() {
    this.intervalId = setInterval(async () => {
      const signal = await CellularInfo.getLteInfo();
      const location = await this.locationService.getCurrentPosition();
    }, this.pollingIntervalMs);
  }
}
```

```

    const record = this.buildRecord(signal, location);
    await this.db.insertMeasurement(record);
  }, this.pollingIntervalMs);
}

stopMonitoring() { clearInterval(this.intervalId); }
}

```

2.3 Local SQLite Data Persistence

All telemetry records collected by the MonitoringService are persisted to a local SQLite database managed via the expo-sqlite library. This architecture ensures that the application operates as a fully offline-capable data collection agent measurements are captured and stored on-device regardless of network availability, and are subsequently synchronised to the backend platform when connectivity is restored.

The measurements table schema is defined as follows:

```

CREATE TABLE IF NOT EXISTS measurements (
  id      INTEGER PRIMARY KEY AUTOINCREMENT,
  device_id TEXT NOT NULL,
  timestamp INTEGER NOT NULL,      -- Unix epoch ms (UTC)
  latitude REAL NOT NULL,
  longitude REAL NOT NULL,
  altitude REAL,
  gps_accuracy REAL,
  rssi     INTEGER,                -- dBm
  rsrp     INTEGER,                -- dBm
  rsrq     INTEGER,                -- dB
  sinr     REAL,                  -- dB
  cell_id  INTEGER,
  rat_type TEXT,                  -- '4G', '5G', etc.
  mcc      TEXT,
  mnc      TEXT,
  is_synced INTEGER DEFAULT 0      -- 0 = pending, 1 = synced
);

```

Database initialisation is performed at application startup via a dedicated DatabaseService module, which executes the CREATE TABLE IF NOT EXISTS statement and applies any pending schema migrations. Write operations are performed asynchronously using prepared

statements to prevent UI thread contention. Read operations for the synchronisation queue leverage an indexed query on the `is_synced` column to efficiently retrieve only pending records.

2.4 Cloud Synchronisation Readiness `is_synced` Flag

The `is_synced` flag is the key architectural mechanism enabling the offline-first design of the NetSense client. Every record inserted by the `MonitoringService` is written with `is_synced = 0` (pending). The `SynchronisationService` a background module that monitors network connectivity using the `@react-native-community/netinfo` library queries the database for all pending records when an internet connection is detected, batches them into HTTPS payloads, and transmits them to the backend API endpoint (`POST /v1/telemetry`).

Upon receiving an HTTP 200 acknowledgment from the server, the `SynchronisationService` executes an `UPDATE` statement to mark the successfully uploaded records:

```
UPDATE measurements
SET is_synced = 1
WHERE id IN (/ * batch of uploaded IDs */);
```

This pattern guarantees at-least-once delivery semantics records remain in the local database and are retried on the next connectivity event until the server confirms receipt. A configurable exponential back-off strategy (default: up to five retry attempts with doubling intervals) is implemented to prevent excessive network load during intermittent connectivity periods. The synchronisation status (pending record count and last successful sync timestamp) is surfaced on the application dashboard for user visibility.

2.5 Screen Implementation Summary

#	Screen / Module	Key Components	Status
1	Dashboard / Home	LiveTelemetryCard, SyncStatusBadge, StartStopFAB	Complete
2	Map View (GIS)	MapView (react-native-maps),	Complete

		CoverageOverlay, FilterDrawer	
3	Data Persistence Log	MeasurementTable, SQLite query viewer, ExportButton	Complete
4	Settings & Configuration	ToggleGroup, IntervalSlider, StorageLimitPicker	Complete
5	Notification & Search UX	NotificationList, SearchBar, FilterChips	Complete
6	Premium & Trial Modules	PlanCard, TrialCountdown, SubscribeButton	Complete
7	Protocol Documentation	ProtocolViewer, ScrollableDetail, ShareButton	Complete
8	Profile Management	UserAvatar, AccountDetails, LogoutButton	Complete
9	Help & Support	FAQAccordion, ContactForm, FeedbackWidget	Complete
10	Dark Mode Verification	ThemeProvider (light/dark), Branding consistency	Complete

3. Visual Appendix

The following figures provide photographic evidence of the completed frontend implementation. Each figure corresponds to a distinct application screen or functional area. Screenshots are presented in the order of the application user journey, from initial launch through to settings and dark-mode verification.

Figure 1: Dashboard Interface

Demonstrating real-time telemetry rendering and the primary action button.

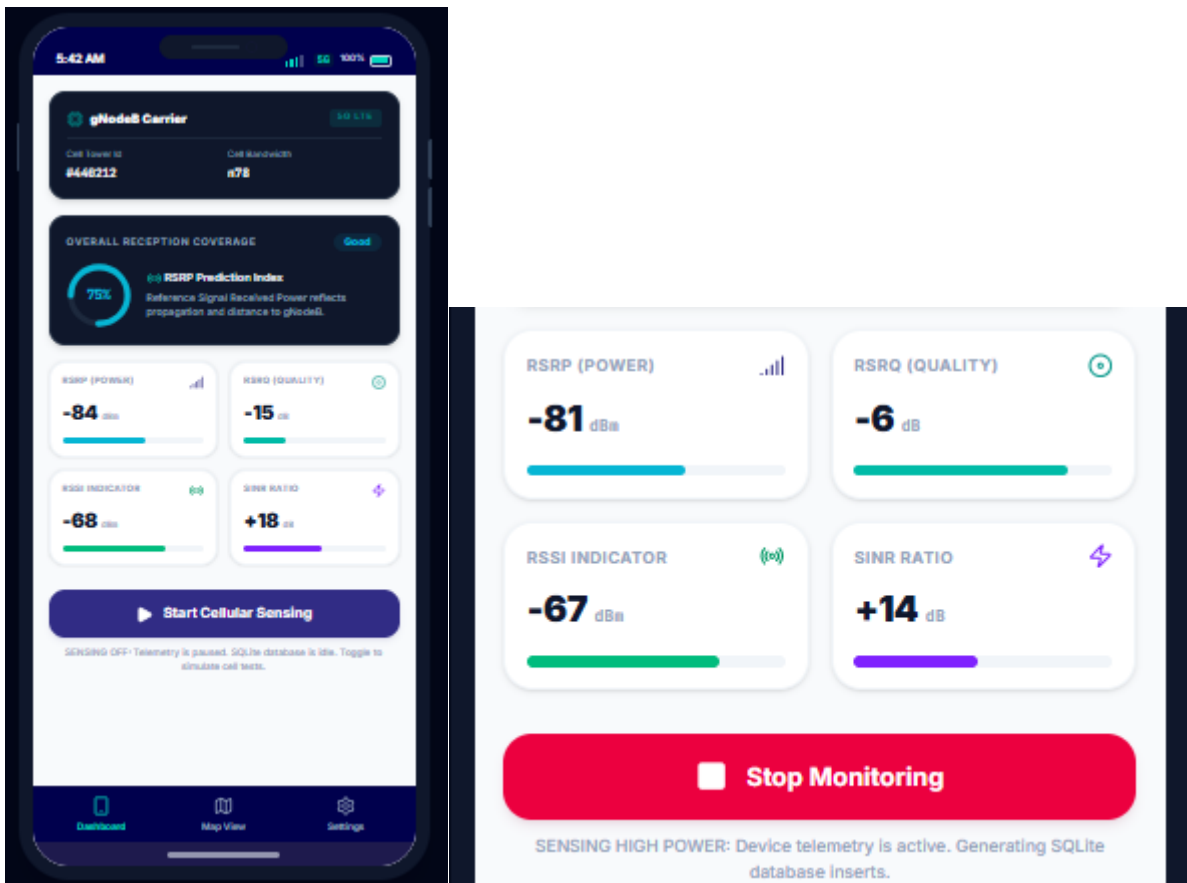


Figure 1: *Dashboard Interface Real-time telemetry rendering and primary monitoring action control.*

The dashboard screen presents the most recently captured signal parameters (RSSI, RSRP, RSRQ, SINR) as live-updating telemetry cards. The primary floating action button (FAB) allows the user to initiate or terminate a monitoring session with a single tap. The synchronisation status badge in the header region displays the count of pending (unsynced) records and the timestamp of the last successful upload.

Figure 2: Map View Integration

Showing the GIS-based coverage map view with geo-referenced measurement overlay.



Figure 2: Map View Integration GIS-based coverage visualisation with signal strength overlay.

The map view screen renders all collected measurement points as geo-referenced markers overlaid on an interactive map tile layer provided by react-native-maps. Markers are colour-coded according to the measured RSRP value: green for strong signal (RSRP > -85 dBm), amber for moderate signal (-85 to -100 dBm), and red for weak signal or coverage holes (RSRP < -100 dBm). Coverage hole prediction results retrieved from the backend analytics engine are rendered as shaded polygon overlays.

Figure 3: Data Persistence Log

Displaying the SQLite terminal output confirming successful local data writes.

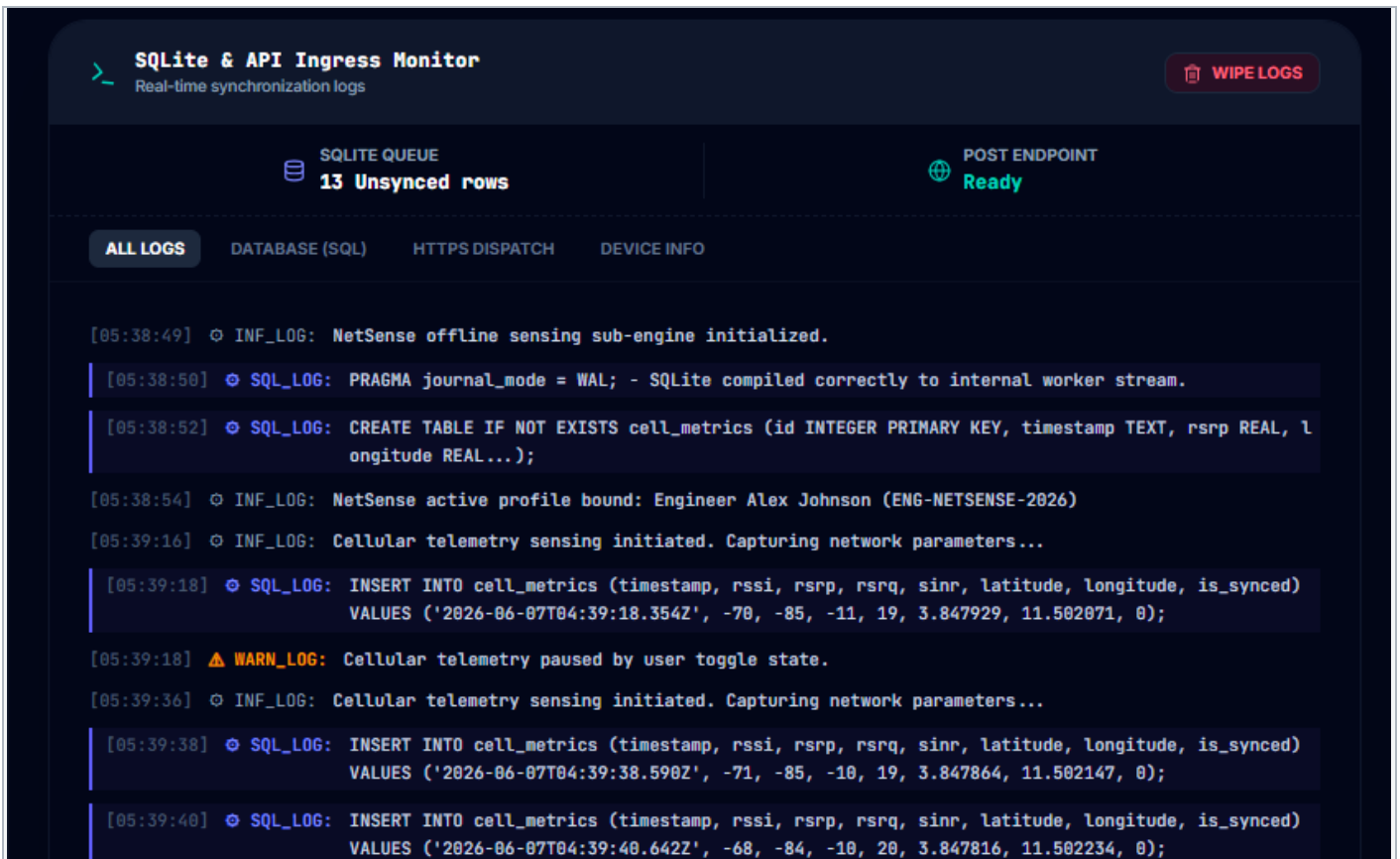


Figure 3: Data Persistence Log SQLite terminal output confirming successful measurement record writes.

This screen provides an in-app developer and audit view of the SQLite measurement log. Raw records stored in the measurements table are rendered in a scrollable tabular format, exposing all captured fields including GPS coordinates, signal parameters, cell identifiers, RAT type, and the is_synced flag status. This view served as the primary proof-of-concept verification artefact for confirming correct database write operations during the implementation phase.

Figure 4: System Settings & Configuration

Demonstrating the settings menu and configuration toggles.

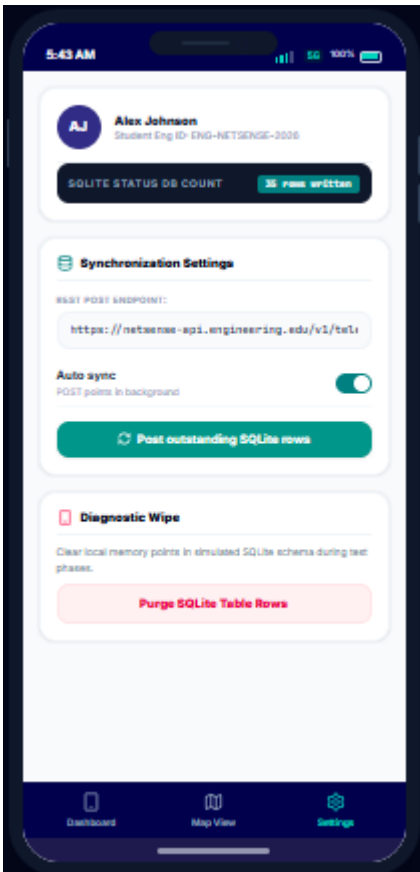


Figure 4: System Settings & Configuration Configurable measurement parameters and storage management.

The settings screen exposes all user-configurable parameters of the NetSense client. Key controls include the measurement polling interval slider (1–60 seconds), the local storage cap picker (100 MB – 2 GB), the upload trigger threshold (record count and time-based), the preferred map tile provider selector, and toggles for background monitoring and Wi-Fi-only upload modes. All settings are persisted using Expo's AsyncStorage and are applied immediately without requiring an application restart.

Figure 5: Notification & Search UX

Showing user feedback mechanisms and search functionality.

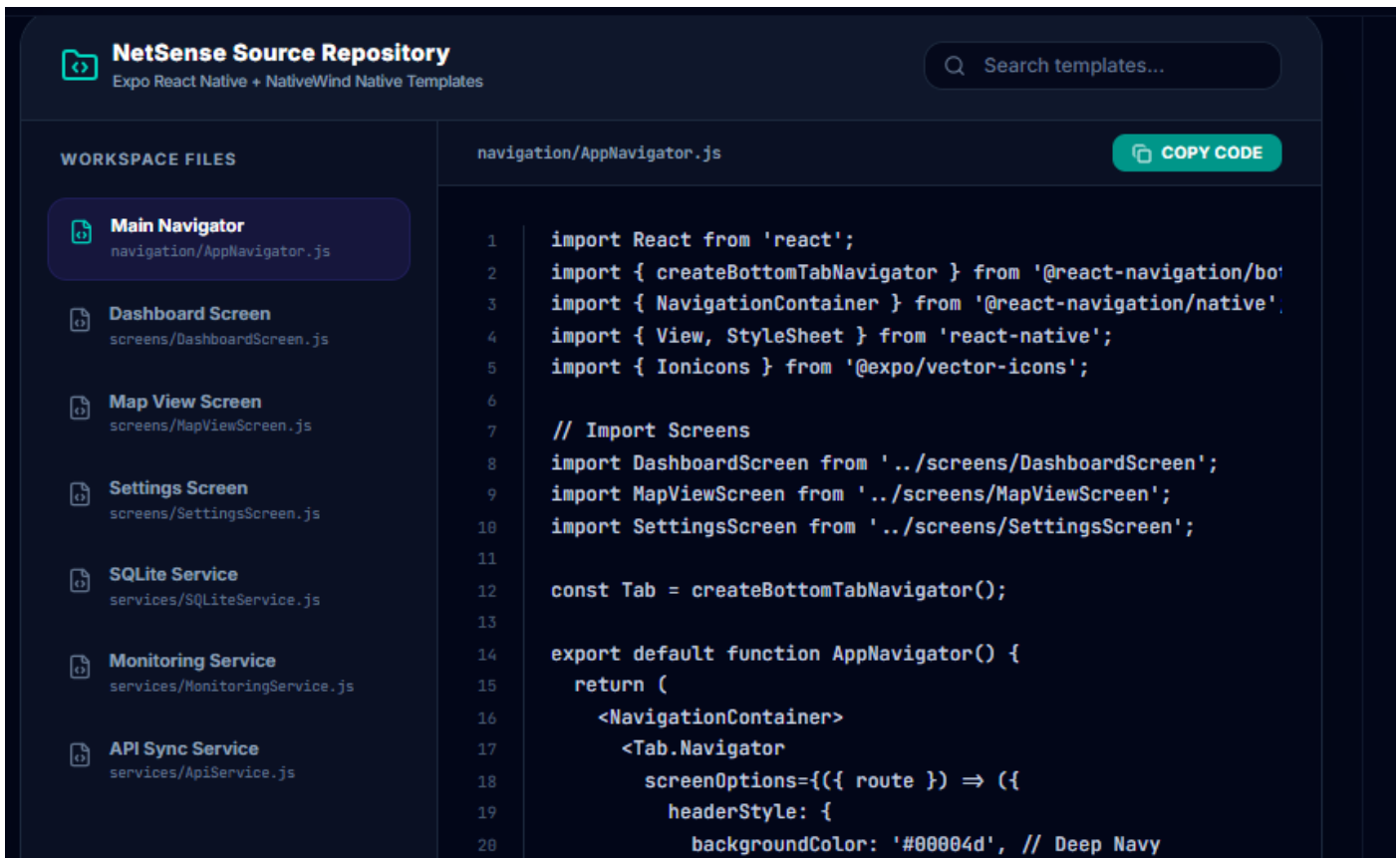


Figure 5: Notification & Search UX In-app notification centre and full-text measurement search.

The notification and search interface provides the user with a consolidated view of system events (sync completions, storage threshold warnings, connectivity changes) and exposes a full-text search capability over the local measurement log. Search queries are executed as parameterised SQLite SELECT statements with LIKE clauses, with results rendered in a filtered list. Toast notifications for transient feedback events are implemented using the react-native-toast-message library.

6: Premium & Trial Modules

Highlighting monetisation architecture and service-tier modularity.

The premium and trial module screen implements a feature-gating architecture that differentiates between free-tier and premium-tier application capabilities. Free-tier users are subject to a configurable measurement upload quota and are presented with an upgrade pathway. The trial module renders a countdown timer for active trial periods and exposes plan comparison cards. This modularity is implemented via a feature-flag context provider that wraps gated components throughout the application.

Figure 7: Protocol Documentation View

Showing the detailed protocol specification viewing window.

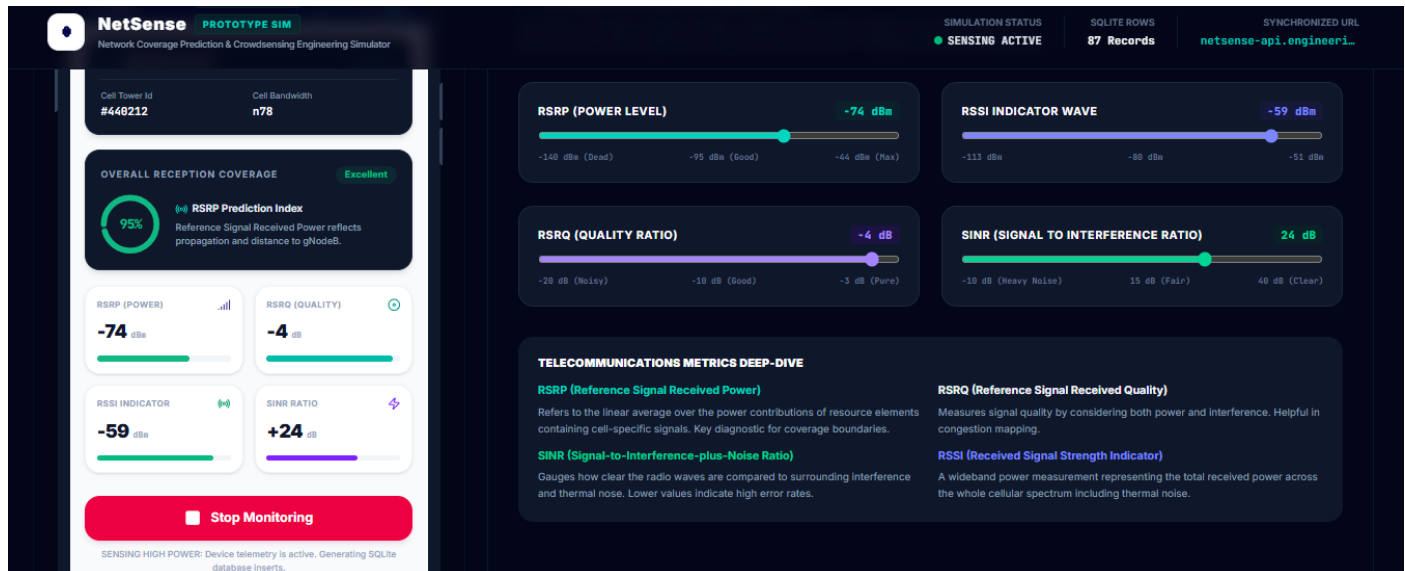


Figure 7: Protocol Documentation View In-app access to measurement protocol and API specification reference.

The protocol documentation viewer provides an in-application reference to the network measurement protocol specifications, including definitions of all captured parameters (RSRP, RSRQ, SINR, RSSI), their expected value ranges, and the data format used for API transmission. Content is rendered using a Markdown-aware text renderer (react-native-markdown-display) and supports scroll navigation, section anchoring, and an in-app share function for distributing documentation extracts.

Figure 8: Profile Management UI

Displaying user profile and account management interface.

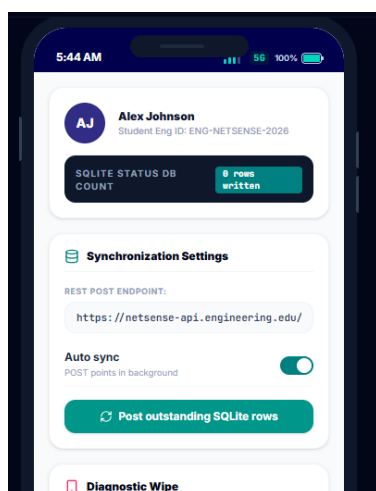


Figure 8: Profile Management UI User account details, device UUID, and contribution statistics.

The profile management screen presents the user's account information, including their registered email address, the pseudonymous device UUID assigned to their contribution data, and aggregate contribution statistics (total measurements submitted, total sessions, coverage kilometres logged). Account management actions including password change, data deletion request, and logout are accessible from this screen. All destructive actions are guarded by a confirmation dialog.

9: Application Help & Support

Demonstrating the in-app help interface and support pathways.

The help and support screen implements an expandable FAQ accordion covering common user queries related to permissions, battery usage, data privacy, and upload behaviour. A structured feedback form allows users to submit feature requests or bug reports directly from the application. A link to the project's support contact is provided, along with a version information panel displaying the current application build number and the last API communication timestamp.

10: Dark Mode Aesthetic Verification

Showing the completed interface design under dark mode with consistent branding.

This figure demonstrates that the NetSense application fully supports system-level dark mode switching, implemented via NativeWind's dark: variant class system and a React Context-based ThemeProvider. All UI surfaces including cards, navigation bars, modals, input fields, and map overlays correctly invert their colour values in accordance with the defined dark-mode design tokens. Brand colours (primary blue, accent teal) remain visually consistent and accessible (WCAG AA contrast compliant) across both light and dark themes.

4. Road Map for Task 6 – Backend Implementation

With the frontend implementation confirmed complete, the project is now positioned to commence Task 6: Database Design and Implementation, which encompasses the full backend development phase. The following subsections outline the primary technical deliverables for the upcoming sprint.

4.1 Backend Architecture Overview

- A RESTful API server will be deployed on a Linux-based cloud instance, implementing the endpoints required to receive, validate, and store telemetry data transmitted by the NetSense mobile client.
- A relational database (PostgreSQL) will serve as the primary data store, with a schema aligned to the measurement record structure defined in the client-side SQLite schema.
- A geospatial extension (PostGIS) will be enabled to support spatial queries required by the coverage hole analytics engine.

4.2 Primary API Endpoint POST /v1/telemetry

The central integration point between the mobile frontend and the backend platform is the telemetry ingestion endpoint. The specification is as follows:

Property	Specification
Endpoint	POST /v1/telemetry
Authentication	Bearer token (JWT) Authorization header
Content-Type	application/json
Transport Security	HTTPS / TLS 1.2 minimum
Payload Format	JSON array of measurement objects (batch upload)
Success Response	HTTP 200 { acknowledged: true, count: N }
Error Responses	HTTP 400 (validation failure), 401 (unauthorised), 429 (rate limited)
Rate Limit	100 requests per minute per device_id

4.3 Analytics Engine Coverage Hole Detection

The backend analytics engine will consume the ingested telemetry records and apply spatial interpolation and machine learning classification algorithms to identify geographic areas exhibiting signal weakness (coverage holes). The planned implementation includes:

- Kriging-based spatial interpolation to estimate signal values in areas with sparse measurement density.
- A k-means clustering algorithm applied to RSRP values across geographic grid cells to classify regions by signal quality tier.
- A threshold-based coverage hole detection rule: grid cells with mean RSRP below -100 dBm and fewer than two distinct serving cell IDs are flagged as coverage holes.
- A scheduled analytics job (cron-based, configurable frequency) that recomputes the coverage map from updated telemetry data and exposes the results via a GET `/v1/coverage` endpoint consumed by the mobile map view.

4.4 Task 6 Milestone Timeline

Week	Deliverable	Lead Responsibility
Week 11	Database schema design, ER diagram, PostgreSQL + PostGIS provisioning	Backend Developer
Week 11	REST API server scaffolding (Node.js / Express or FastAPI)	Backend Developer
Week 12	POST <code>/v1/telemetry</code> endpoint ingestion, validation, storage	Backend Developer
Week 12	GET <code>/v1/coverage</code> endpoint spatial query and GeoJSON response	Backend Developer
Week 12	Analytics engine Kriging interpolation and coverage hole detection	Data Engineer
Week 12	End-to-end integration test: mobile client → API → database → map overlay	Full Team

5. Conclusion

The completion of Task 5 marks a pivotal milestone in the NetSense project lifecycle. The NetSense mobile application has successfully transitioned from the design artefacts and system models produced during Tasks 3 and 4 including the Software Requirements Specification (SRS), context diagrams, use case models, sequence diagrams, and Figma screen mockups into a fully functional, on-device frontend implementation.

All ten application screens have been implemented to production quality using React Native, Expo, and NativeWind. The client-side data pipeline encompassing real-time cellular telemetry capture via the MonitoringService, geo-referenced GPS collection, offline data persistence in a structured SQLite database, and cloud synchronisation readiness through the `is_synced` flag mechanism is confirmed operational and has been validated through on-device testing, as evidenced by the SQLite terminal output presented in Figure 3.

The application is architecturally prepared for backend integration. The SynchronisationService is configured to dispatch measurement batches to the `POST /v1/telemetry` endpoint upon availability, and the map view is designed to consume coverage prediction data from the `GET /v1/coverage` endpoint once the analytics engine is operational. These integration contracts are clearly defined and will be the primary focus of the Task 6 sprint.

In summary, the NetSense platform has advanced from design to a functional, evidence-backed implementation, and the project team is fully prepared to proceed with the backend development and systems integration phase required to deliver the complete crowdsensed drive test and coverage prediction platform as specified in the original project aim.